

Education

Nanjing University 2023.09 - 2026.06 (expected)  
M.S. in Computer Science and Technology | PASCAL Lab. Advisor: Li | Research: PL and program analysis.  
Teaching Assistant: Principles and Techniques of Compilers (Spring 2024)

Beihang University 2019.09 - 2023.06  
B.S. in Computer Science and Technology | GPA: 3.84/4.00; awarded National Scholarship, Outstanding Graduate, and First Prize in the 2021 NSCSCC Compilation System Design Competition.  
Teaching Assistant: Programming in Practice (Fall 2020), Object-Oriented Design and Construction (Fall 2021, Spring 2022)

Work Experience

Tencent 2026.05 - Present  
Security Department III Security Technology (Qingyun Program)

NVIDIA 2025.02 - 2025.10  
OCG (Optimizing Code Generator) team GPU Compiler LLVM Backend Intern

- Merged NVIDIA GPU graphics and NVVM memory-instruction vectorizers, aligning with LLVM upstream:
  - Encoded multi-address graphics memory instructions to reuse LLVM memory-vectorizer logic;
  - Added graphics-instruction alias analysis by tracing BasicAA and SCEVAA behavior on new intrinsics;
  - Improved LLVM alignment inference by solving GCDs over SCEV expressions for loop induction variables;
  - Implemented 10+ graphics-memory vectorization optimizations, including irregular access sequence padding, raising vectorization success rate by 40%;
- Maintained the graphics compiler and fixed bugs in Vectorization, SCEV, and related optimizations;

Rust Foundation Fellowship Program 2024.09 - 2025.09  
Rust Foundation Fellowship (approximately 20 fellows globally) Project Fellow

- Rust-lang Organization member (rust-analyzer-contributors-team) and maintainer of rust-lang/rust-analyzer, the official Rust IDE; ranked in the top 1% of contributors, triaged issues, reviewed PRs, and helped maintain rust-clippy;
- Shipped control-flow highlighting, snapshot-test updates, and bug fixes across code understanding, code generation, and related IDE workflows;
- Implemented a NEON SIMD version of the Unicode line-breaking module, achieving a 6.5x speedup on ARM;
- v0.3.1992 incident response: Within 3 hours, replaced a flawed dependency-graph search algorithm and resolved a severe resource-exhaustion bug discovered four hours after release.

Projects

VIDE VIDE  
Modern IDE for Chip Frontend Design · Master’s Thesis Project Rust / SystemVerilog

- Built a SystemVerilog semantic analysis framework and IDE infrastructure for modern chip-design workflows;
- Designed an incremental analysis IR and pass pipeline on top of an incremental computation architecture, enabling accurate analysis without full recomputation;
- Delivered 10+ SystemVerilog IDE features, including navigation, refactoring, and completion, with millisecond-level latency via incremental semantic analysis; supports VS Code, Emacs, and other LSP editors;

Ailurus roife/ailurus  
Programming Language and Toolchain Design Exploration · Personal Interest Project Rust

- Based on Martin-Löf type theory; supports dependent types, dependent pattern matching, inductive datatypes, equality proofs, and NbE;
- Implements typeclass-based ad-hoc polymorphism, operator overloading, and a module system for namespaces;
- Experimental platform for co-designing modern programming-language toolchains, including compilers and IDEs, for better efficiency and maintainability.

## 🔗 Open Source Contributions

- 🛡️ Maintainer of [rust-lang/rust-analyzer](#), the official Rust IDE / language server; contributed to the [Rust compiler](#), [clippy](#), [rustup](#), and [rust-mode](#)
  - Other OSS: [LLVM](#), [vscode-clangd](#), [lua-language-server](#), [autocxx](#), [goldmark](#), [tree-sitter-moonbit](#), and [more on GitHub](#).
- 

## 📋 Skills

- **Programming Languages:** Proficient in C, C++, Rust, Java, Python, JavaScript/TypeScript, Verilog/SystemVerilog, and EmacsLisp; also used Ruby, Swift, OCaml, Haskell, and Coq;
  - **Programming Language Theory:** Formal semantics, type theory, computation models, and automata; experience with proof assistants including Coq;
    - **(Type Systems)** Theory and implementation of Hindley-Milner, subtyping, System F, and dependent types;
    - **(Static Analysis)** 4 YoE with theory and implementation of abstract interpretation, data-flow / control-flow / pointer analysis, and CodeQL/Datalog interpreters.
  - **Compiler Design:** 3 YoE. End-to-end compiler development from parsing to codegen; familiar with **IRs** such as SSA, DBI;
    - **(Language Implementation)** Object-oriented / functional languages; bidirectional type checking and related features;
    - **(IDE Development)** 3 YoE with rust-analyzer / clangd, **LSP**, and **incremental computation** architecture;
    - **(Compiler Optimization)** 3 YoE with middle-end / backend optimization, including Mem2Reg, GVN, RegAlloc, and List Scheduling; familiar with **LLVM** internals, LLVM-IR, and MLIR;
  - **Application Development:** Ruby on Rails, Django, SwiftUI; PostgreSQL, Redis; Docker, CI/CD, and DevOps.
  - **Development Environment:** Proficient in Emacs / VS Code on macOS / Linux; skilled at using generative AI tools to improve efficiency.
- 

## 📁 Misc

- **Talks:** *Exploring rust-analyzer: from Incremental Computation to Modern IDE* (RustChinaConf 2025)